



Universidad de las Fuerzas Armadas – ESPE

Departamento de Ciencias de la Computación

Carrera de Ingeniería de Software

Análisis y Diseño de Software - NRC 30724

Tema

Patrones de Diseño

Integrantes

Cuzco Alex

Domínguez Pablo

Alvarado Dylan

Profesor/a: Ing. Jenny Robalino

Fecha: 21 de Mayo del 2026

1. Introducción

El diseño de software orientado a objetos busca soluciones que no solo resuelvan problemas inmediatos, sino que también permitan la reutilización y flexibilidad del sistema ante cambios futuros. Los patrones de diseño representan soluciones probadas para problemas recurrentes en el desarrollo. Entre ellos, el patrón Flyweight optimiza el uso de memoria compartiendo objetos de grano fino, mientras que los patrones comportamentales como Observer y Strategy gestionan las interacciones entre objetos. En el presente informe se analizará las características esenciales, ventajas, desventajas y la aplicación de estos patrones en un ejemplo real del CRUD de estudiantes

2. Objetivo General

Diseñar e implementar un sistema CRUD de estudiantes que sea altamente eficiente en el uso de recursos y fácilmente extensible, mediante la integración de patrones de diseño estructurales y comportamentales para optimizar la gestión de datos y la respuesta de la interfaz de usuario.

2.1. Objetivos específicos

- Investigar y comprender el rol de los patrones de diseño en el desarrollo de software orientado a objetos mediante la implementación práctica de los patrones Flyweight, Observer y Strategy, evaluando su impacto en la eficiencia del sistema, la reutilización del código y la capacidad de adaptación ante nuevos requerimientos.
- Analizar e implementar patrones de diseño estructurales y comportamentales, aplicándolos en un sistema CRUD de estudiantes para comprender cómo optimizan el uso de recursos, mejoran la organización del código y facilitan la extensibilidad del software ante futuros cambios.

3. Patrones de Diseño

3.1. Patrones Estructurales

3.1.1. Flyweight (Peso ligero)

Descripción

El patrón Flyweight es una solución de diseño orientada a objetos cuya intención es "usar el compartimiento para soportar grandes cantidades de objetos de grana fina de manera eficiente". Un flyweight es un objeto

compartido que puede usarse en múltiples contextos de manera simultánea, comportándose como un objeto independiente en cada contexto, siendo indistinguible de una instancia no compartida.

El Flyweight es un patrón de diseño estructural. Se clasifica como "Object Structural", es decir, un patrón estructural de objetos, dado que su propósito es organizar la composición de objetos para lograr estructuras más eficientes en memoria.

Características

El patrón Flyweight debe aplicarse cuando se cumplen todas las siguientes condiciones:

- La aplicación usa una gran cantidad de objetos.
- Los costos de almacenamiento son altos debido a la cantidad de objetos.
- La mayor parte del estado de los objetos puede hacerse extrínseco.
- Muchos grupos de objetos pueden reemplazarse por relativamente pocos objetos compartidos una vez removido el estado extrínseco.
- La aplicación no depende de la identidad de los objetos; dado que los flyweights son compartidos, las pruebas de identidad retornarán verdadero para objetos conceptualmente distintos.

Ventajas

El principal beneficio es el ahorro de almacenamiento. Los ahorros dependen de: la reducción en el número total de instancias gracias al compartimiento, la cantidad de estado intrínseco por objeto, y si el estado extrínseco es calculado o almacenado. El mayor ahorro se obtiene cuando los objetos usan cantidades sustanciales de ambos estados y el extrínseco puede calcularse en lugar de almacenarse. Como ejemplo práctico, un documento con 180,000 caracteres requirió solo 480 objetos de caracteres al aplicar el patrón

Desventajas

Los flyweights pueden introducir costos en tiempo de ejecución asociados con la transferencia, búsqueda y/o cálculo del estado extrínseco, especialmente si antes era almacenado como estado intrínseco. Sin embargo, estos costos se compensan con los ahorros en espacio.

Aplicación en el CRUD de un Estudiante

En un sistema CRUD de estudiantes, datos como carrera, facultad, ciudad de origen o año de ingreso son compartidos por muchos estudiantes (estado intrínseco). El nombre, número de identificación y calificaciones específicas serían estado extrínseco, únicos para cada estudiante. Una `CarreraFlyweightFactory` podría retornar una instancia compartida de "Ingeniería de Sistemas" en lugar de crear un nuevo objeto por cada estudiante inscrito en esa carrera, reduciendo significativamente el uso de memoria cuando hay miles de registros.

3.2. Patrones Comportamentales

3.2.1. Observer (Observador)

El patrón Observer, también conocido como Observer Pattern o patrón observador, es uno de los patrones de diseño de software más populares, incluido en el libro clásico de 1994 *Design Patterns: Elements of Reusable Object-Oriented Software*. “Este patrón se encarga de separar el estado de un objeto de su representación visual, permitiendo que existan múltiples formas de mostrar esa información al mismo tiempo, a través de una dependencia de uno a varios objetos”. Su funcionamiento se basa en que cualquier objeto puede registrarse como observador de otro objeto principal, llamado sujeto. Cuando este sujeto cambia de estado, todos los observadores registrados son notificados automáticamente y se actualizan para reflejar ese cambio de la forma más sencilla y rápida posible. Entre las principales características que destacan a este patrón de diseño están: :

- Se utiliza en situaciones donde se requieren **múltiples formatos de despliegue** para la información de un estado (por ejemplo, un gráfico y una tabla simultáneamente).
- El objeto que contiene el estado no necesita mantener información sobre los formatos específicos de despliegue que se están utilizando.
- Involucra un **Sujeto** (Subject), que mantiene a los observadores y los notifica, y **Observadores** (Observers), que reaccionan a la notificación mediante un método de actualización (Update).
- El modelo UML muestra que el Sujeto tiene métodos para adjuntar (Attach), desadjuntar (Detach) y notificar (Notify) a los observadores.

Ventajas

- **Desacoplamiento:** Permite que el objeto principal sea independiente de cómo se visualice o quién lo esté observando.
- **Actualización en tiempo real:** Garantiza que todas las representaciones de los datos estén sincronizadas automáticamente ante cambios en el origen.
- **Flexibilidad:** Es posible añadir nuevos observadores o formatos de presentación sin necesidad de modificar el objeto que mantiene el estado.

Desventajas

- Aunque las fuentes no listan desventajas de forma explícita en una tabla, se puede inferir del diseño que el Sujeto debe gestionar una lista de observadores, lo que puede añadir complejidad al sistema si el número de observadores crece significativamente.
- Existe el riesgo de que un cambio de estado provoque una cascada de actualizaciones que afecte el rendimiento si no se controla adecuadamente.

El patrón Observer resulta muy útil en el desarrollo de software por varias razones. Primero, si se necesita modificar alguno de los observadores, los demás componentes del sistema prácticamente no se ven afectados, lo que hace que adaptar o rediseñar partes del código sea mucho más sencillo.

Además, las clases creadas con este patrón se pueden reutilizar fácilmente en otros proyectos, ya que separan los comportamientos compartidos de los datos específicos de cada objeto, reduciendo el trabajo de mantenimiento con el tiempo. Por último, como la lógica principal del programa queda separada de la parte visual, se pueden hacer cambios en la interfaz de usuario, como corregir errores o mejorar el diseño, sin arriesgar que los datos del sistema se vean comprometidos o alterados.

Aplicación en el CRUD estudiantes:

Actualmente nuestro proyecto está trabajando con:

- RepositorioEstudiante almacena los datos en memoria.
- FormularioCrudEstudiante interactúa con el usuario.
- ControlEstudiante procesa la lógica.

Ahora, aplicando el Patrón de Diseño Observer sería:

- RepositorioEstudiante actuaría como **Subject**.
- La interfaz FormularioCrudEstudiante sería un **Observer**.

En este sentido, cuando se agregue, edite o elimine un estudiante, el repositorio notificará automáticamente a la interfaz para refrescar:

- tablas,
- listas,
- contadores,
- mensajes de estado.

3.2.2. Strategy

Descripción

Tiene como objetivo definir una familia de algoritmos, encapsular cada uno de ellos y hacerlos intercambiables. Su propósito fundamental es permitir que el algoritmo varíe de forma independiente de los clientes que lo utilizan.

Características

- Encapsula algoritmos o comportamientos en clases separadas.
- Reduce el uso excesivo de estructuras condicionales.
- Facilita la reutilización y mantenimiento del código.

Aplicación en el CRUD estudiante

El patrón Strategy puede aplicarse para separar diferentes tipos de validaciones del sistema.

ControlEstudiante contiene validaciones como:

- ID positivo.
- Edad positiva.
- Nombre solo con letras.

Con Strategy, cada validación podría convertirse en una estrategia independiente:

- ValidacionID
- ValidacionEdad
- ValidacionNombre

Ventajas

- Mejora la organización y modularidad del código.
- Permite reutilizar algoritmos o validaciones.
- Facilita las pruebas individuales de cada estrategia.
- Hace el sistema más flexible ante cambios futuros.

Desventajas

- Incrementa la cantidad de clases en el proyecto.
- Puede volver el diseño más complejo para proyectos pequeños.
- Requiere que el desarrollador conozca correctamente las interfaces.

4. Conclusiones

5. Recomendaciones

6. Bibliografia

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.